

Algorithmic Attacks

Cryptography

CRYPTOGRAPHIC ATTACKS

A difficult aspect of analysing cryptographic protocols is answering the question: What constitutes an attack?

- Are both authentication and secrecy assured?
- Is it possible to impersonate one or more of the parties?
- Is it possible to interject messages from an earlier exchange (replay attack)?
- What tools can an attacker deploy?
- *If any key is compromised, what are the consequences?

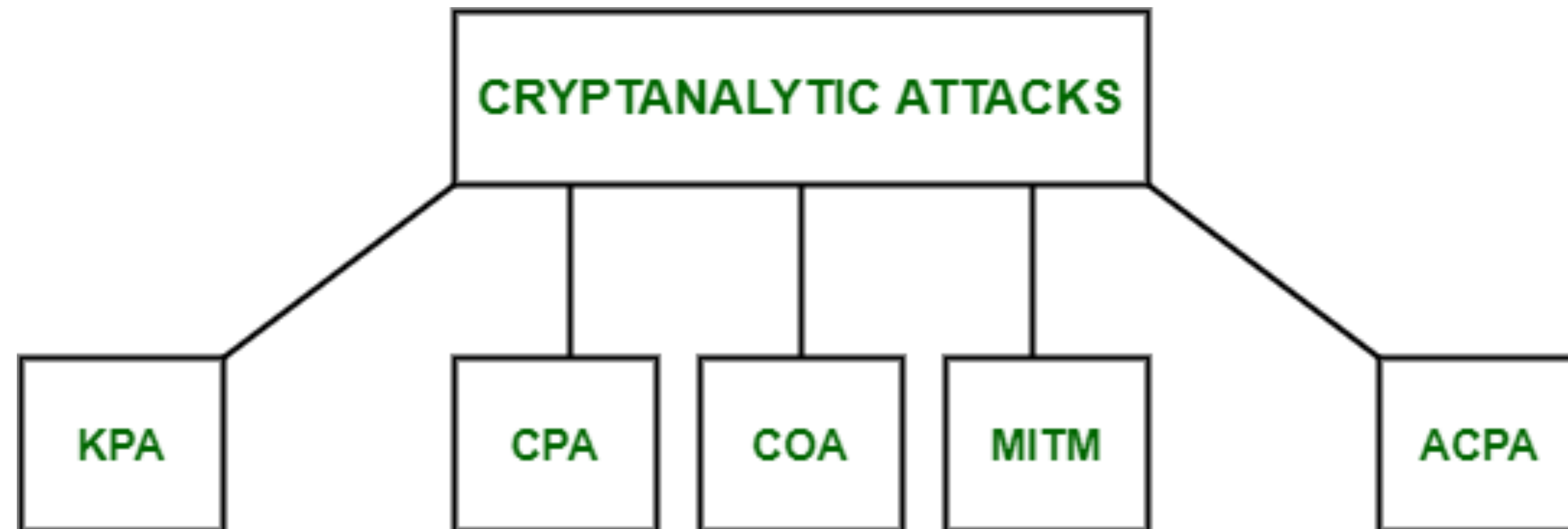
Is the last question really fair?

Some protocols have been in use for years before someone noted a significant vulnerability.

Cryptography

CRYPTOGRAPHIC ATTACKS - Cryptanalysis

- Practice of analysing and breaking cryptography



Cryptography

CRYPTOGRAPHIC ATTACKS

Two of the most common types of attacks on cryptography include algorithm attacks and collision attacks.

Known ciphertext attacks. The goal of the attacker is to get the encryption key or the plaintext. Attackers sometimes can use sophisticated statistical tools to analyze the ciphertext and discover a pattern in the ciphertexts. This type of attack is called a known ciphertext attack or ciphertext-only attack, because all that is known is the ciphertext, though it can still reveal clues that may be mined.

Known plaintext attacks. The attacker has access to the plaintext and the corresponding cipher text and tries to get the key. Example: linear cryptanalysis

Chosen plaintext attack: A scenario in which the attacker has the ability to choose plaintexts and to view their corresponding encryptions—ciphertexts. Encrypting without knowing the key is accomplished via an “encryption oracle,” or a device that encrypts without revealing the key. e.g., using Differential cryptanalysis

Chosen ciphertext and adaptive chosen ciphertext: Chosen ciphertext attacks mirror chosen plaintext attacks. The difference is that the cryptanalyst chooses the ciphertext to be decrypted. This attack is usually launched against asymmetric cryptosystems, where the cryptanalyst may choose public documents to decrypt that are signed (encrypted) with a user's public key.

Birthday Attack: Basically, a threat actor would be forced to try all possible combinations until a collision was found. However, a statistical phenomenon called the birthday attack makes it easier. The birthday attack is named after the birthday paradox. The name is based on the fact that in a room with 23 people or more, the odds are greater than 50% that two will share the same birthday. Many find this counterintuitive, and the birthday paradox illustrates why many people's instinct in probability (and risk) is wrong. You are not trying to match a specific birthday (e.g., yours); you are trying to match any birthday.

Collision attacks: Two files having the same digest is known as a collision. A collision attack is an attempt to find two input strings or file of a hash function that produce the same hash result.

Cryptography

CRYPTOGRAPHIC ATTACKS

Brute force: A brute-force attack generates the entire key-space, which is every possible key. Given enough time, the plaintext will be recovered.

Man-in-the-middle attack: attacker interposes himself between two parties and pretends to each to be the other.

Side-channel attacks: Side-channel attacks use physical data to break a cryptosystem, such as by monitoring CPU cycles or power consumption used while encrypting or decrypting.

Downgrade attack. Because new hardware and software are introduced frequently, they often include backwards compatibility so that a newer version can still function with the older version. However, in most instances, the newer version must revert to the older and less secure version. In a downgrade attack, an attacker forces the system to abandon the current higher security mode of operation and instead “fall back” to implementing an older and less secure mode. The threat actor can then attack the weaker mode.

Differential cryptanalysis: This type of attack involves comparing pairs of plaintexts and their corresponding ciphertexts to find patterns in the encryption algorithm. It can be effective against block ciphers with certain properties.

Attacks Based on Misconfigurations. Most breaches of cryptography are the result of incorrect choices or mis-configurations of the cryptography options, known as misconfiguration implementation. Selecting weak algorithms, such as DES or SHA-1, should be avoided since they are no longer secure. Many cryptographic algorithms have several configuration options, and unless careful consideration is given to the options during setup, the cryptography may be improperly implemented. Careless users can also weaken cryptography if they choose SHA-256, for example, when a much stronger SHA3-512 is available through a simple menu choice.

Cryptography

CRYPTOGRAPHIC KEY CHALLENGES

Weak keys: A key is essentially just a random number – the longer and more random it is, the more difficult it is to crack. The strength of the key should be appropriate for the value of the data it is protecting and the period of time for which it needs to be protected. The key should be long enough for its intended purpose and generated using a high-quality (ideally certified) random number generator (RNG).

Incorrect use of keys: Each key should be generated for a single, specific purpose (i.e. the intended application and algorithm) – if it is used for something else, it may not provide the expected or required level of protection.

Re-use of keys: Improper re-use of keys in certain circumstances can make it easier for an attacker to crack the key.

Non-rotation of keys: If a key is over-used (e.g. used to encrypt too much data), then it makes the key more vulnerable to cracking,

Inappropriate storage of keys: Keys should never be stored alongside the data that they protect (e.g. on a server, database, etc.), as any exfiltration of the protected data is likely to compromise the key also.

Cryptography

CRYPTOGRAPHIC KEY CHALLENGES

Inadequate protection of keys: Even keys stored only in server memory could be vulnerable to compromise. Where the value of the data demands it, keys should be encrypted whenever stored and only be made available in unencrypted form within a secure, tamper-protected environment and even (in extreme cases) kept offline. There have been a number of vulnerabilities that could expose cryptographic keys in server memory including Heartbleed, Flip Feng Shui and Meltdown/Spectre.

Insecure movement of keys: It is often necessary to move a key between systems. This should be accomplished by encrypting (“wrapping”) the key under a pre-shared transport key (a key encryption key, or KEK), which may be either symmetric or asymmetric. Where this is not possible (e.g. when sharing symmetric transport keys to bootstrap the system), the key should be split into multiple components that must then be kept separate until being re-entered into the target system (and then the components are destroyed).

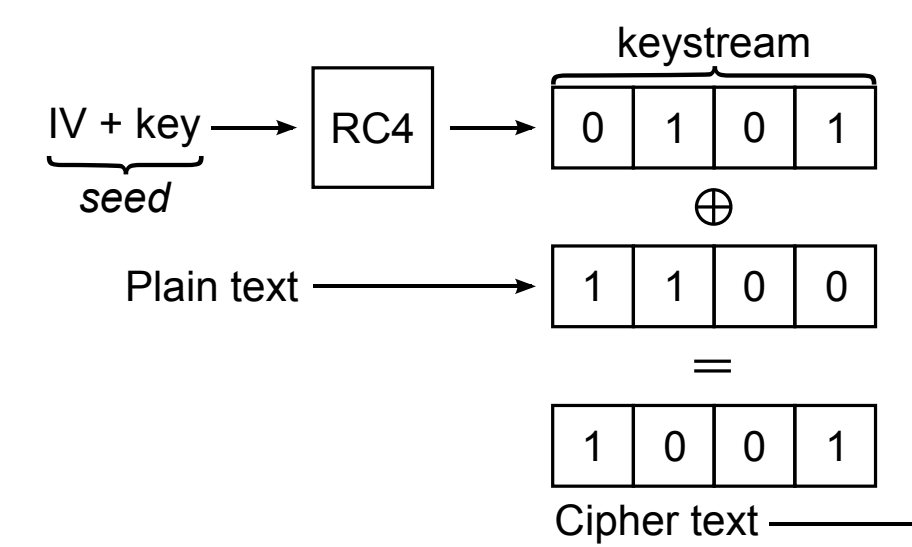
Non-destruction of keys: Keys should be destroyed (i.e. securely deleted, leaving no trace) once they have expired, unless explicitly required for later use (e.g. to decrypt data). This removes the risk of accidental compromise at some future date.

Manual key management processes: The use of manual key management processes, using paper or inappropriate tools such as spreadsheets and accompanied by manual key ceremonies, can easily result in human errors that often go unnoticed and may leave keys highly vulnerable.

EXAMPLES

Cryptography

CRYPTOGRAPHIC ATTACKS - Examples



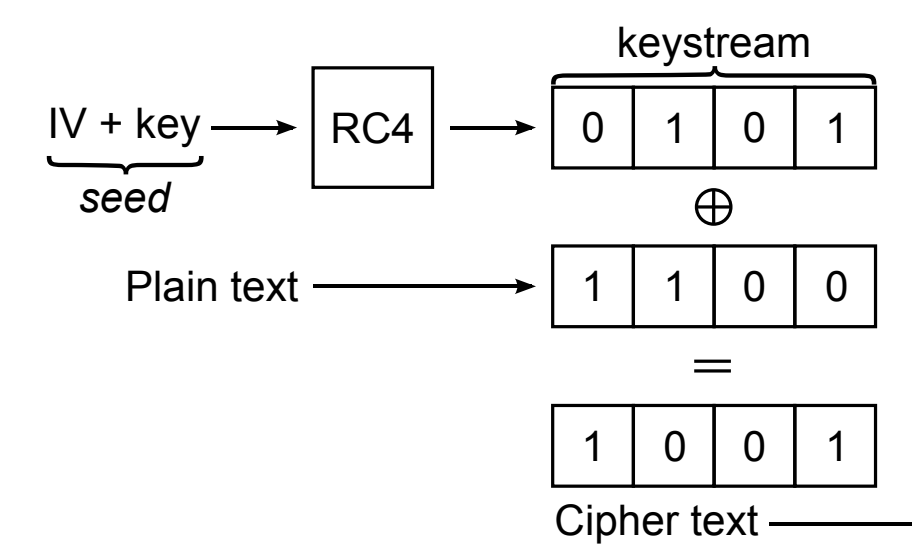
Wired Equivalent Privacy (WEP) was one of the first security protocols used in early Wi-Fi networks to protect data from eavesdropping and unauthorised access. However, WEP had several significant weaknesses that made it vulnerable to attacks, leading to data breaches. Here's an overview of these weaknesses and how attackers were able to crack WEP keys:

Weaknesses in WEP:

- 1. Static Key Management:** WEP relied on a static and unchanging encryption key that was shared among all devices on the network. This made it susceptible to long-term attacks.
- 2. Short Key Length:** WEP used a relatively short key length of either 64 bits or 128 bits. With advancements in computing power, these key lengths became insufficient for security.
- 3. Poor Initialization Vector (IV) Handling:** WEP used a 24-bit IV to introduce randomness into the encryption process. However, the limited IV space meant that IVs were reused, making it easier for attackers to capture multiple packets encrypted with the same IV.
- 4. Weak Encryption Algorithm:** WEP used the RC4 stream cipher for encryption, which had known vulnerabilities, especially when the same key and IV were reused.
- 5. Lack of Authentication:** WEP focused on encryption but did not provide a robust authentication mechanism, making it possible for unauthorised users to join the network.

Cryptography

CRYPTOGRAPHIC ATTACKS - Examples



Wired Equivalent Privacy (WEP) was one of the first security protocols used in early Wi-Fi networks to protect data from eavesdropping and unauthorised access. However, WEP had several significant weaknesses that made it vulnerable to attacks, leading to data breaches. Here's an overview of these weaknesses and how attackers were able to crack WEP keys:

Weaknesses in WEP:

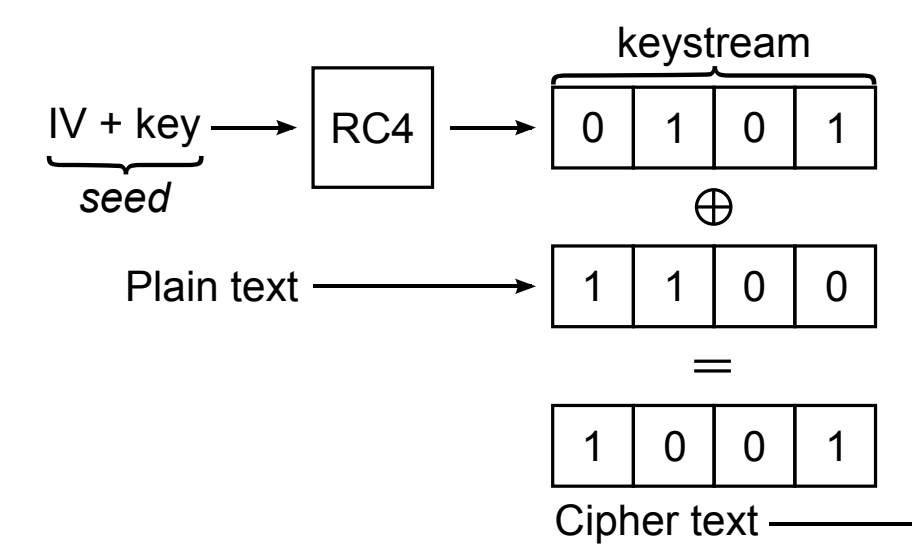
- 1. Static Key Management:** WEP relied on a static and unchanging encryption key that was shared among all devices on the network. This made it susceptible to long-term attacks.
- 2. Short Key Length:** WEP used a relatively short key length of either 64 bits or 128 bits. With advancements in computing power, these key lengths became insufficient for security.
- 3. Poor Initialization Vector (IV) Handling:** WEP used a 24-bit IV to introduce randomness into the encryption process. However, the limited IV space meant that IVs were reused, making it easier for attackers to capture multiple packets encrypted with the same IV.
- 4. Weak Encryption Algorithm:** WEP used the RC4 stream cipher for encryption, which had known vulnerabilities, especially when the same key and IV were reused.
- 5. Lack of Authentication:** WEP focused on encryption but did not provide a robust authentication mechanism, making it possible for unauthorised users to join the network.

Attacks on WEP:

- 1. IV Attacks:** Attackers could capture a large number of encrypted packets, each with a unique IV, in a process known as IV collection. By analyzing these packets, attackers could discover the keystream, allowing them to decrypt future communications.
- 2. KoreK Attacks:** The KoreK attack suite introduced a set of tools to exploit weak IVs and discover the WEP key. KoreK attacks significantly improved the efficiency of WEP key cracking.
- 3. Fragmentation Attacks:** Attackers could use fragmentation attacks to obtain known plaintext. With enough known plaintext, they could deduce parts of the keystream and recover the encryption key.
- 4. Chop-Chop Attacks:** In a Chop-Chop attack, attackers manipulated the ciphertext and observed the behaviour of the access point. This allowed them to determine the correctness of the guessed plaintext, assisting in the key recovery process.
- 5. Packet Injection:** Attackers could inject malicious packets into the network and observe how the access point responded. This could reveal information about the encryption key, making it easier to crack.

Cryptography

CRYPTOGRAPHIC ATTACKS - Examples



Wired Equivalent Privacy (WEP) was one of the first security protocols used in early Wi-Fi networks to protect data from eavesdropping and unauthorised access. However, WEP had several significant weaknesses that made it vulnerable to attacks, leading to data breaches. Here's an overview of these weaknesses and how attackers were able to crack WEP keys:

Weaknesses in WEP:

- 1. Static Key Management:** WEP relied on a static and unchanging encryption key that was shared among all devices on the network. This made it susceptible to long-term attacks.
- 2. Short Key Length:** WEP used a relatively short key length of either 64 bits or 128 bits. With advancements in computing power, these key lengths became insufficient for security.
- 3. Poor Initialization Vector (IV) Handling:** WEP used a 24-bit IV to introduce randomness into the encryption process. However, the limited IV space meant that IVs were reused, making it easier for attackers to capture multiple packets encrypted with the same IV.
- 4. Weak Encryption Algorithm:** WEP used the RC4 stream cipher for encryption, which had known vulnerabilities, especially when the same key and IV were reused.
- 5. Lack of Authentication:** WEP focused on encryption but did not provide a robust authentication mechanism, making it possible for unauthorised users to join the network.

Attacks on WEP:

- 1. IV Attacks:** Attackers could capture a large number of encrypted packets, each with a unique IV, in a process known as IV collection. By analyzing these packets, attackers could discover the keystream, allowing them to decrypt future communications.
- 2. KoreK Attacks:** The KoreK attack suite introduced a set of tools to exploit weak IVs and discover the WEP key. KoreK attacks significantly improved the efficiency of WEP key cracking.
- 3. Fragmentation Attacks:** Attackers could use fragmentation attacks to obtain known plaintext. With enough known plaintext, they could deduce parts of the keystream and recover the encryption key.
- 4. Chop-Chop Attacks:** In a Chop-Chop attack, attackers manipulated the ciphertext and observed the behaviour of the access point. This allowed them to determine the correctness of the guessed plaintext, assisting in the key recovery process.
- 5. Packet Injection:** Attackers could inject malicious packets into the network and observe how the access point responded. This could reveal information about the encryption key, making it easier to crack.

Consequences:

These weaknesses and attacks made it relatively straightforward for attackers to crack WEP keys, leading to unauthorised access to Wi-Fi networks and potential data breaches. In some cases, attackers were able to gain access to sensitive information, such as login credentials, emails, and other data transmitted over the network.

As a result, the security community recognised the urgent need for a more robust Wi-Fi security protocol, leading to the development and adoption of Wi-Fi Protected Access (WPA) and its later iterations, which offered stronger encryption, improved key management, and more robust security mechanisms to replace the vulnerable WEP. Today, using WEP is considered highly insecure, and it is strongly recommended to upgrade to more secure Wi-Fi security protocols like WPA3.

Cryptography

CRYPTOGRAPHIC ATTACKS - Examples

POODLE (Padding Oracle On Downgraded Legacy Encryption):

1. **Vulnerability Description:** POODLE was a vulnerability that affected the SSLv3 protocol. It allowed attackers to recover plaintext data from encrypted communications by exploiting the way SSLv3 used padding for block ciphers like CBC (Cipher Block Chaining).

1. Mitigation:

- o **Protocol Deprecation:** To address POODLE, one of the primary strategies was to deprecate and disable the vulnerable SSLv3 protocol. This effectively removed the insecure protocol from use.
- o **TLS 1.2 Adoption:** Migrating to TLS 1.2 and higher, which use secure cipher suites and offer improved cryptographic security, was a fundamental response to mitigate POODLE.
- o **Cipher Suite Changes:** Changing cipher suites to prefer those that use AEAD (Authenticated Encryption with Associated Data) modes, such as AES-GCM (Galois/Counter Mode), which do not require padding and are less susceptible to padding oracle attacks.
- o **Fallback SCSV (Signaling Cipher Suite Value):** The introduction of the TLS_FALLBACK_SCSV extension helped prevent downgrade attacks. It allowed servers to detect when clients attempted to downgrade the protocol version and reject such connections.

Cryptography

CRYPTOGRAPHIC ATTACKS - Examples

POODLE (Padding Oracle On Downgraded Legacy Encryption):

1. **Vulnerability Description:** POODLE was a vulnerability that affected the SSLv3 protocol. It allowed attackers to recover plaintext data from encrypted communications by exploiting the way SSLv3 used padding for block ciphers like CBC (Cipher Block Chaining).

1. **Mitigation:**

- o **Protocol Deprecation:** To address POODLE, one of the primary strategies was to deprecate and disable the vulnerable SSLv3 protocol. This effectively removed the insecure protocol from use.
- o **TLS 1.2 Adoption:** Migrating to TLS 1.2 and higher, which use secure cipher suites and offer improved cryptographic security, was a fundamental response to mitigate POODLE.
- o **Cipher Suite Changes:** Changing cipher suites to prefer those that use AEAD (Authenticated Encryption with Associated Data) modes, such as AES-GCM (Galois/Counter Mode), which do not require padding and are less susceptible to padding oracle attacks.
- o **Fallback SCSV (Signaling Cipher Suite Value):** The introduction of the TLS_FALLBACK_SCSV extension helped prevent downgrade attacks. It allowed servers to detect when clients attempted to downgrade the protocol version and reject such connections.

ROCA Attack (Return of Coppersmith's Attack):

- **Vulnerability Description:** The ROCA attack was a cryptographic vulnerability that affected the RSA key generation algorithm in cryptographic libraries, including the widely used Trusted Platform Module (TPM) firmware. This vulnerability allowed attackers to factor RSA keys and recover private keys from the public keys. It was a significant threat to systems that relied on RSA for encryption and authentication.
- **Mitigation:**
 - o **Key Length Increase:** One of the primary mitigation strategies in response to the ROCA attack was to increase the minimum recommended RSA key length. Longer key lengths, such as 2048 bits or 3072 bits, were strongly recommended to resist factorization attempts.
 - o **Vulnerable Key Detection:** Many security tools and services were developed to help organizations detect vulnerable RSA keys generated with faulty libraries and software. These tools enabled organizations to identify and replace keys that were potentially compromised.
 - o **Firmware and Software Updates:** Hardware manufacturers and software developers released updates and patches to address the ROCA vulnerability in their cryptographic libraries and implementations. These updates corrected the flawed key generation algorithms and improved the security of generated RSA keys.
 - o **Random Number Generation Improvement:** To prevent future vulnerabilities of a similar nature, there was an increased focus on improving random number generation in cryptographic libraries and key generation processes.
 - o **Secure Implementation Practices:** Best practices for secure key generation, secure coding, and cryptographic library development were reinforced to ensure that cryptographic implementations are resistant to factorization attacks and vulnerabilities like ROCA.

Cryptography

CRYPTOGRAPHIC ATTACKS - Examples

BEAST (Browser Exploit Against SSL/TLS):

1. **Vulnerability Description:** BEAST was an attack that targeted the CBC mode used in SSL/TLS protocols. It allowed attackers to decipher parts of secure SSL/TLS-encrypted communications by exploiting predictable initialization vectors and the way the CBC mode operated.
1. **Mitigation:**
 - o **TLS 1.1/1.2 Adoption:** Migrating to newer versions of TLS that included countermeasures against BEAST. In particular, TLS 1.1 and 1.2 implemented a form of record splitting to disrupt the attack.
 - o **Changing Cipher Suites:** Many server administrators and clients changed their cipher suite preferences to prioritize RC4-based cipher suites, which were immune to the BEAST attack.
 - o **TLS Renegotiation: Disable and Patch:** Disabling or properly patching TLS renegotiation mechanisms to prevent certain types of attacks that were related to BEAST.
 - o **Transport Layer Security Session Resumption:** Implementing session resumption mechanisms that did not reuse the same initialization vectors, reducing the potential for exploitation.